

Full-Stack Medical Monitoring and Document Health Platform – Technical & Feature Guide

Overview

This guide presents a comprehensive plan for a **Medical Monitoring and Document Health Platform** built with **Go** (backend), **Angular** (frontend), **SQL** (PostgreSQL or MySQL), and free/open-source tools. The platform combines an electronic medical records system (for doctors and patients) with a knowledge base health monitor. Key features include: document “health” scoring (freshness, versioning, conflict detection), doctor/patient portals for managing records, an admin dashboard, automated reminders via email/SMS, visual analytics dashboards, smart alerts (e.g. outdated records, missing diagnoses), real-time chat, robust security (RBAC, encryption), and document similarity detection using AI. The following sections detail the functional features, technical architecture, data design, libraries/services, security practices, and an incremental implementation plan (with sprints through April 22).

Feature Overview

1. Knowledge Base Health Monitoring

The platform will incorporate all features from the original **Knowledge Base Health Monitor**:

- **Document Health Scoring:** Each knowledge base document is assessed for “health” – e.g. recency of updates, correctness, and consistency with other documents. A scoring algorithm can combine metrics like last updated time, usage, and conflict checks to quantify document health. Documents will have a visible health indicator (e.g. percentage or color code).
- **Versioning and Changelogs:** Every knowledge article is version-controlled. Updates increment a version number, and older versions are retained or archived for reference. This ensures historical knowledge isn’t lost and allows tracking of changes over time ¹. The system may use lightweight version tags (e.g. v1.0, v1.1 for minor edits, etc.) and log update dates in the document metadata ¹.
- **Freshness Tracking:** The platform flags stale documents. Administrators can define a “staleness” threshold (e.g. 6 or 12 months since last update). The system highlights documents exceeding this age without updates ². For example, a knowledge article not updated in over 6 months might be marked as needing review ². Admins can receive periodic reports of approaching or overdue documents, prompting content updates (quarterly health checks) ³ ⁴.
- **Conflict & Similarity Detection:** Using AI-based text embeddings, the system automatically detects overlapping or contradictory content among knowledge documents. When two documents cover similar topics, the platform can flag potential duplicates or conflicts. Best practices from knowledge management suggest consolidating overlapping entries and archiving or labeling outdated duplicates ⁴. For instance, if two guidelines conflict (e.g. dosing instructions), the system would alert knowledge admins to reconcile them. Implementation details for this feature are covered in [Document Similarity & AI Integration](#) below.
- **Analytics for KB Health:** An admin-facing dashboard will display metrics like number of documents, average health score, % of documents updated recently vs stale, count of flagged conflicts, etc. This “Knowledge Base Health” dashboard helps track maintenance efforts ⁵ ⁶. It can also show usage stats (views, search success rate) to gauge which articles are effective or need improvement.

Overall, these features ensure the medical knowledge base (e.g. internal guidelines, help articles) remains **up-to-date, accurate, and consistent**, thus improving its reliability for practitioners. Periodic audits and the automated scoring/alerts will facilitate proactive content maintenance.

2. Doctor and Patient Portals

The platform provides role-specific portals for **Doctors** and **Patients**, with an account system to securely segregate data. Key capabilities include:

- **Doctor Portal Features:**

- **Patient Records Management:** Doctors can create and update electronic health records for each patient. This includes documenting visits/encounters with notes (SOAP notes or similar), diagnoses, and treatment plans. Doctors can view complete medical histories of their patients (past visits, chronic conditions, etc.).
- **E-Prescriptions:** Doctors are able to upload or generate prescriptions for medications. These prescriptions are linked to the patient's record and are visible to the patient. The system supports printing or downloading prescriptions in a standardized format (PDF) and could be extended for e-prescribing standards (e.g. integration with pharmacies) ⁷.
- **Notes & Attachments:** Doctors can log clinical notes for each appointment (including uploading images or files like lab result PDFs). Each note is timestamped and versioned if edited later. A **file upload history** is maintained per patient – doctors (and patients, if permitted) can see all files uploaded for that patient, with dates and descriptions, ensuring nothing is overlooked.
- **View Medical History:** Doctors have a consolidated view of a patient's profile: demographics, past diagnoses, active medications, allergies, recent lab results, etc. This helps in informed decision-making. They can also filter or search within a patient's records (e.g. find all past notes mentioning "diabetes").
- **Scheduling & Calendar:** The doctor portal includes an appointment calendar. Doctors can schedule new appointments or see upcoming ones, with patient info and reason for visit. A calendar view (daily/weekly) helps manage workload and avoid conflicts. This calendar can be shared with patients for booking or integrated with reminders. (The admin can also use scheduling data for analytics, see Admin section.)
- **Communication & Collaboration:** Doctors can communicate with patients or other doctors through the platform's messaging features (see [Chat & Messaging](#)). They can also leave internal comments on a patient's record (e.g. tagging another doctor for a second opinion, viewable only to staff).

- **Patient Portal Features:**

- **Profile and Medical Data Access:** Each patient can log in to view their own records and health data. They can see past and current **prescriptions**, including dosage instructions and start/end dates. They can also view **lab test results** or reports uploaded by doctors (after the doctor has reviewed/released them). Sensitive data is accessible only to the patient and authorized doctors (enforced via access controls).
- **Medication Reminders:** Patients receive automated reminders for their medications. Upon receiving a prescription, the patient can be reminded via email/SMS when it's time to take a dose or refill (based on the schedule set by the doctor). For example, an SMS reminder can be sent daily at prescribed times, using an SMS API on a free tier. (We discuss implementation in [Notifications & Reminders](#).)
- **Progress Tracking:** Patients can log or view tracked health metrics – for instance, they might input daily blood pressure readings or symptoms, which doctors can later review. A simple

interface for patients to **upload test results** (e.g. if they got an external lab test or used a home device) is provided. These uploads are added to their record (pending doctor approval if needed). The patient can then see all their uploaded files/history as well.

- **Appointments & Calendar:** Patients can view upcoming appointments with their doctors and receive reminders. Optionally, allow patients to request or book appointments through the portal (subject to doctor approval). This ties into the scheduling system – e.g., the patient sees available slots and books one.
- **Secure Messaging with Providers:** Patients can send messages or questions to their doctor through a secure chat interface, instead of phone or email. This can be asynchronous messaging (like a secure email) or real-time chat if the doctor is online. It improves engagement and quick clarifications. (This uses the same chat system described later.)
- **Download/Print Records:** For usability, patients should be able to download or print certain items: e.g. a summary of their medical record, lab results, or a prescription. The system can generate PDF exports for these. This feature increases patient empowerment and portability of data (useful for second opinions or emergency situations).

These doctor/patient features turn the platform into a basic **Electronic Medical Records (EMR) system**. Notably, the system will support typical EMR functions like managing **prescriptions, lab results, notes, reminders, secure messaging, and scheduling** ⁸ ⁹. We will ensure that doctors see only their patients' data, and patients see only their own data, enforcing strict privacy.

3. Admin Role and System Management

An **Admin** role has oversight over the entire system. Admin users have system-wide access to manage users, monitor usage, and configure settings. Key admin features include:

- **User & Role Management:** Admins can create or deactivate accounts for doctors and patients, reset passwords, and assign roles. They also manage roles/permissions (for example, promoting a doctor to also have admin privileges, or managing which features are enabled for patients). Role-Based Access Control (RBAC) underpins this (discussed in Security).
- **System Dashboard:** A central admin dashboard provides real-time monitoring of system health and usage. This includes metrics such as: number of active users, number of new patient sign-ups, daily logins, etc. It also shows counts of key entities (total patients, total doctors, documents in knowledge base, etc.). If any subsystem (e.g. the Go API or database) has health endpoints, their status can be displayed.
- **Knowledge Base Health Analytics:** As part of the knowledge base monitor, admins see the **KB health dashboard** mentioned earlier. For example, the dashboard might show “85% of documents updated in last 6 months” or list the top 5 documents needing review ² ⁴. Admins can drill down to see which documents are flagged stale or conflicting.
- **Patient Data Analytics:** Visual summaries of patient data can be provided. For instance, the admin could see aggregated metrics: how many prescriptions were issued this month, how many appointments took place, average number of active medications per patient, etc. This can be helpful for clinic management.
- **Audit Logs:** The system logs critical actions (especially due to medical data sensitivity). Admins can review logs of user activity, such as logins, record edits, document deletions, etc. This is important for security auditing and compliance.
- **System Configuration:** Admin settings allow adjusting global configurations – e.g. toggling features (enable/disable patient self-registration), setting the email/SMS gateway credentials, defining the knowledge base “staleness” threshold (in days), scheduling automated jobs (like what time reminders are sent daily), etc.

- **Alerts and Error Monitoring:** Admins receive **smart alerts** (discussed in next section) about important events. For example, if an automated job finds a patient record missing a follow-up, or if a scheduled reminder failed to send, the admin is notified. The admin interface should highlight any such alerts until resolved.

In essence, the admin role ensures **smooth operation and oversight**. They can monitor both the **clinical usage (doctors/patients)** and the **knowledge base content**. With these tools, an admin can maintain quality (up-to-date documents, active user engagement) and respond quickly to issues.

4. Notifications & Reminders (Email/SMS)

Automated notifications are a crucial feature for improving adherence and engagement. The platform will support sending out emails or SMS messages for: medication reminders, appointment reminders, and possibly alerts like abnormal lab results. We will utilize free-tier or open services for this:

- **Medication Reminders:** When a doctor prescribes a medication and sets a schedule (e.g. “take once daily at 8 AM for 14 days”), the system will schedule reminder notifications. Using a background job scheduler, the Go backend can queue daily messages to the patient (email or SMS based on preference). For example, a patient gets an SMS: “Reminder: Take medicine X (10mg) now, as prescribed.” This improves compliance with treatment. We can implement this with an open-source scheduling library in Go (such as `robfig/cron`) to trigger at specific times. For SMS, a service like **Twilio** can be used – Twilio’s free trial allows limited SMS sending (e.g. messaging verified numbers) which may suffice for initial development. Twilio trial supports on the order of ~100 SMS in total and up to 35 SMS/day for certain endpoints ¹⁰. Alternatively, email reminders can use a free tier of **SendGrid** (which offers ~100 emails/day free). Both options ensure no cost for low-volume notifications. (*Note: Patients would opt-in to such reminders as per consent requirements.*)
- **Appointment Reminders:** The system sends patients a reminder a day (or configurable time) before an upcoming appointment. This can reduce no-shows. This is typically an email with appointment date/time and any preparation notes, or an SMS “Reminder: You have an appointment with Dr. Smith tomorrow at 3:00 PM.” The admin can configure how far in advance to remind and via which channel. As with meds, these can be scheduled on the backend when an appointment is created. Using email/SMS free-tier services similarly as above (SendGrid, Twilio, etc.) to deliver the notifications. In fact, OpenEMR and similar systems list patient appointment notification via email/SMS as a standard feature ⁹, validating our approach.
- **System Alerts via Email:** Aside from patient-facing reminders, the system can send internal alerts to admins or doctors. For example, if a lab result is uploaded that is marked “critical” (out-of-range values), an email alert to the doctor could be sent immediately. Another case: an admin receives an email if the knowledge base has a document that’s expired or if the disk space is running low (the KB monitor example showed sending emails when storage thresholds exceeded ¹¹ ¹²). We will integrate an email API (or SMTP server) for such messages. Many transactional email services have free quotas (for instance, Mailgun’s free tier allows a certain number of emails/month).

For implementing email, we might integrate a Go library (like `gomail` or an SMTP client) to send through a free SMTP service. For SMS, Twilio’s Go SDK can be used to call the API (with the understanding of trial limits). **All credentials** (API keys) will be stored in configuration (not hard-coded) and handled securely.

Scheduling Service: The platform will likely include a background worker or cron-like service to handle these notifications. We can run a scheduler within the Go backend (since Go can handle concurrent schedulers), or use a lightweight task queue. The scheduling system will query upcoming reminders

each day and send out messages accordingly. If using Twilio trial, note that trial accounts can only send SMS to verified recipient numbers and prepend “(Trial)” to messages ¹⁰ – for a production scenario we’d upgrade or use an open alternative.

By leveraging such free-tier services and careful scheduling, we provide robust **email/SMS reminder functionality at no or minimal cost** to the project.

5. Visual Dashboards & Smart Alerts

The platform emphasizes **visualization of data** for better insight, and **smart alerts** to catch important issues proactively:

- **Visual Dashboards:** We will build interactive dashboards for different user roles:
- **Patient Health Dashboard:** Each patient, upon logging in, can see a simple health dashboard – e.g. a chart of their recent lab results (trends of blood sugar, blood pressure over time), a progress tracker for any ongoing treatment (percentage of medication doses taken on time, etc.), and upcoming schedule (next appointment, days left on current prescription). These visuals encourage patient engagement.
- **Doctor’s Dashboard:** A doctor’s home screen can show an overview of their practice: number of patients under their care, how many appointments today (with a quick link to each), any unread messages from patients, and alerts for any patient requiring attention. For instance, if test results came in that are critical or a patient hasn’t responded to a follow-up, it could be highlighted. Workload charts (like appointments per week or tasks pending) help doctors manage time.
- **Admin Analytics Dashboard:** As mentioned, admins have dashboards for system-wide metrics, knowledge base health, etc. This likely includes charts like “New patients per month”, “Most common prescriptions”, or system performance metrics. We will use open-source **charting libraries** to implement these dashboards on the Angular frontend. For example, **Chart.js** (with an Angular wrapper like ng2-charts) can produce bar graphs, line graphs for trends, pie charts for distribution of document states, etc., all free to use. Another option is **Apache ECharts** or **D3.js** for more custom visuals. Given Angular’s ecosystem, using a ready Angular chart component (e.g. **NGX-Charts**) can speed up development.

These dashboards will be interactive and filterable (e.g. admin can filter KB stats by category, doctor can filter their patients by risk level, etc.). The data will be fetched via REST API calls from the Go backend (which will aggregate data via SQL queries).

- **Smart Alerts:** The system will generate **intelligent alerts** for certain conditions to improve care quality and data accuracy:
- **Outdated Medical Records:** For example, if a patient hasn’t had an annual check-up or a particular important lab test in a long time, the system could alert the doctor or patient. Another case: if a patient’s problem list (diagnoses) hasn’t been updated in a long time despite multiple visits, it might indicate something was missed. These rules can be configured (like “flag patients with >1 year since last full blood panel”). The alert could be a notification on the doctor dashboard or even an automated message to the patient to schedule an appointment.
- **Missing Diagnoses or Follow-ups:** The platform can analyze data to find inconsistencies such as *test results with no recorded follow-up*. For instance, if a patient had an abnormal lab result uploaded (e.g. high cholesterol) but the record doesn’t show any diagnosis or plan addressing it, the system flags this. This is akin to clinical decision support: ensuring nothing “falls through the cracks.” In healthcare, failure to follow up on test results is a known issue – incorporating EMR alerts is recommended by safety guidelines ¹³ . Our system could, for example, list all

“unresolved findings” for the doctor to review. Similarly, if a doctor notes a possible condition in free text but doesn’t add it to the structured diagnosis list, an NLP-based check could prompt them.

- **Knowledge Base Alerts:** Alerts also apply to the KB side – e.g., *“Document X has not been updated in over a year”*, or *“Two documents on ‘Hypertension treatment’ appear to conflict”*. These would alert knowledge admins to take action (update or reconcile the docs). In practice, a **stale content alert** might be shown next to the document title in the KB interface ² or emailed to the content owner. Conflict alerts might be shown after a similarity scan (using the AI system below).
- **System Health Alerts:** If any system issues occur (database down, high load, etc., assuming we set up health checks), the admin can be alerted via email. This ensures high availability and quick response to technical problems.

The **“smart” aspect** means many alerts are triggered by automated analysis (rules or AI) rather than manual entry. We will implement rule-based checks for straightforward conditions (dates, missing fields) and leverage AI for more complex pattern recognition (like conflicting documents). The alert generation can run as a scheduled job (e.g. nightly batch processing to scan records and docs for issues) and/or real-time triggers (e.g. when a test result is entered, immediately check if it’s abnormal with no follow-up plan).

All alerts will be delivered through the appropriate channel: in-app notifications (badge icons or dashboard highlights), and optionally email/SMS for critical ones. This multi-channel approach ensures important alerts are not missed, aligning with patient safety recommendations (for example, one study suggests using EMR alerts to ensure patients are contacted about critical results ¹³).

6. Chat & Messaging

Effective communication is vital in a healthcare setting. The platform will include a **secure messaging system** that supports: (a) direct messages or chat between patients and doctors, and (b) comments or discussion threads attached to specific records or documents. The functionality includes:

- **Patient-Doctor Messaging:** Patients can send a message to their doctor (e.g. to ask a question about a prescription or report a minor side effect). The doctor can reply when available. This works similarly to a chat or email thread. To implement this, we can use a **websocket** for real-time updates if both are online, or fall back to polling/new-message indicators if not. An open-source library like **Gorilla WebSocket** in Go can handle real-time communication easily. Messages will be stored in the database (with sender, receiver, timestamp, and content), ensuring a history is maintained for medical record purposes. The UI in Angular will present a chat window or a message center. We will ensure this communication is encrypted and authenticated (only the intended patient and doctor can read their conversation). OpenEMR’s patient portal lists “Secure Messaging and Chat” as a feature ⁸ , and we aim to provide similar capability.
- **Comments on Visits/Documents:** In addition to direct chat, the system supports commenting within context. For example, a doctor viewing a patient’s record can leave a comment on a specific visit entry (“Referred to specialist – awaiting feedback”) which could be visible to other doctors in the team. Patients might be allowed to comment or ask a question on a visit summary (“I forgot, should I take this pill with food?”), which notifies the doctor. Similarly, knowledge base documents could have a comment thread for collaborators to discuss updates or flag issues. These comments would be implemented as records in a comments table linking to the relevant entity (visit ID or document ID).
- **Notifications for Messages:** When a new message or comment is posted, the intended recipient(s) get notified. For instance, a doctor gets an in-app notification if a patient messages

them, and possibly an email alert if they haven't logged in recently. Patients likewise get an email notice if a doctor replies (to prompt them to check the portal). This ensures timely responses.

- **Real-time Considerations:** For real-time chat, we will incorporate a presence indicator (show if doctor is online for chat). Given our stack, one approach is to have the Angular app open a WebSocket connection to the Go backend for the chat component. The Go server can use an in-memory hub (or a pub/sub system) to forward messages to the correct recipient if online. This is feasible with Go's concurrency model. If implementing from scratch is complex, we might consider using an open-source messaging backend like **Matrix** or **XMPP**, but those could be overkill. A simpler custom implementation is sufficient for our scope.
- **Audit and Privacy:** All communications are part of the medical record from a legal standpoint, so we archive them. The admin (or a compliance officer) might need access to these in case of disputes, so admin privileges could include viewing chat transcripts if needed (with proper audit logs of such access). We will also implement **end-to-end encryption** for messaging if possible, or at least ensure TLS in transit and encryption at rest for sensitive message content (see Security).

Overall, the messaging system will **streamline doctor-patient communication**, reducing phone calls and improving documentation. It also adds a collaboration layer for maintaining the knowledge base (authors can discuss within the platform rather than offline). This feature increases the platform's usability and user engagement significantly.

7. Security, Privacy & Access Control

Security is paramount since the platform handles Protected Health Information (PHI) and sensitive documents. We will employ a multi-layered security approach:

- **Role-Based Access Control (RBAC):** We define distinct roles (at minimum: Admin, Doctor, Patient) and enforce access rules accordingly. **Patients** can only access their own records and certain knowledge base documents (if the knowledge base is internal, perhaps patients have access to a subset like educational materials). **Doctors** can access records of patients they manage (and knowledge articles), but not other doctors' patients (unless there's a referral mechanism). **Admins** have global access but will use it only for support tasks. We will likely implement RBAC in the backend using a policy library like **Casbin** or a custom middleware. For example, Casbin can let us define policies such as "doctors can GET/POST on `/patients/{id}/records` only if {id} is in their patient list" in a config. Casbin supports role hierarchies (e.g. an admin role can inherit all doctor permissions) ¹⁴. This approach cleanly separates authorization logic from code. OpenEMR also supports fine-grained per-user access controls and role-based menus ¹⁵, indicating such RBAC is a standard practice in medical systems.
- **Authentication:** We will implement secure authentication, likely with **JWT (JSON Web Tokens)** for the API. Users log in (username/password), the Go server verifies credentials (stored as bcrypt-hashed passwords in the DB), and issues a JWT token that Angular stores (probably in memory or secure HTTP-only cookies to prevent XSS). Each request then includes this token for auth. Alternatively, we could use cookie-based sessions with server-side session store; JWT is simpler for stateless backend usage and mobile-friendliness. Passwords will be **hashed using bcrypt or Argon2** (via a Go crypto library) – never stored in plaintext. We'll enforce strong password policies. Optionally, we can include 2FA (using an authenticator app or SMS OTP via Twilio) as a future enhancement for high security.
- **Data Encryption:** We plan to encrypt sensitive data both **in transit** and **at rest**. In transit, all client-server communication will be over **HTTPS** with TLS, which is non-negotiable especially for PHI (HIPAA mandates encryption in transit ¹⁶). For at rest: the database can contain highly sensitive fields (e.g. patient SSN, medical history text). We will use database-level encryption or

app-level encryption for certain fields. For instance, the OpenEMR system explicitly “Ability to Encrypt Patient Documents” ¹⁵ – we can follow suit by encrypting files or notes stored in the DB using keys. We might use a symmetric encryption (like AES) for any file attachments or particularly sensitive notes. The key management will be important – possibly derived from an environment secret or using a secure vault service. If using PostgreSQL, we could enable the **pgcrypto** extension to encrypt columns. Alternatively, store encrypted blobs from the app side.

- **Audit Trails and HIPAA Compliance:** The system will maintain logs of data access. Each time a patient record is viewed or edited by a doctor, it can record who accessed what and when. This is critical for privacy audits. Similarly, any export of data (like if a PDF is generated for printing) could be logged. These measures align with HIPAA’s requirement of record access tracking. The encryption of PHI at rest, while *addressable* under HIPAA, is highly recommended ¹⁷ ¹⁸ – implementing it provides an extra layer of defense if the database were compromised.
- **Secure Development Practices:** We will guard against common web vulnerabilities. Examples: Use prepared statements or an ORM to avoid SQL injection; validate and sanitize all inputs on the server; use Angular’s built-in XSS protections for output; implement CSRF protection if we use cookies; limit file upload types and scan for malware (particularly because patients can upload files – we might integrate a virus scan on uploads using an open-source scanner like ClamAV). We’ll also enforce **content security policy** on the web app to prevent loaded scripts.
- **Data Privacy & Partitioning:** Each patient’s data is logically isolated. In the database, we might have separate schemas or at least use foreign keys to ensure no data crossover. If multi-tenant (e.g. multiple clinics) were in scope, we’d have a tenant ID on records to segregate them. Role-based menu restrictions (like OpenEMR’s approach ¹⁵) will ensure users don’t even see UI elements they’re not allowed to use.
- **Backups and Failover:** While not directly asked, a note on data safety: we will set up automated backups for the database (free solutions include periodic dump to secure cloud storage). Encryption keys for at-rest data should be backed up separately and securely.
- **Testing Security:** We will perform thorough testing, possibly using open-source tools to scan for vulnerabilities. For instance, run OWASP ZAP against the web app, etc., to catch any oversight.

By adhering to these practices, the platform will maintain a high level of security. **RBAC and encryption are explicitly implemented** to protect patient data ¹⁵. Additionally, compliance considerations (HIPAA in healthcare) are met by using encryption for PHI and auditing access ¹⁸. Users will only be able to access what they should, and all data transmissions are secured. This gives users (and administrators) confidence in the platform’s safety for sensitive health information.

8. Document Similarity & AI Integration

To achieve advanced features like **document similarity search and conflict detection**, we will integrate an AI component, using either open-source Large Language Models (LLMs) or Azure’s Retrieval Augmented Generation (RAG) services (or a hybrid of both). The goals are to: (a) find semantically similar documents in the knowledge base (for deduplication or search), and (b) detect potential contradictions between documents or between a document and new information.

Open-Source Approach: We can leverage **open-source embedding models** and vector search libraries entirely within our stack (ensuring zero recurring cost). The plan:

- Use a text **embedding model** to convert documents (or their sections) into high-dimensional vectors that capture semantic meaning. There are many open models (e.g. **SentenceTransformers** like *all-MiniLM* or *mpnet*) that perform well. We can either call a Python service for this or use a Go library that enables embeddings. A promising option is the **kelindar/search** Go library, which wraps **llama.cpp** to run language models and generate embeddings in Go ¹⁹ ²⁰. This library supports loading compact models (like MiniLM-L6-v2) and computing embeddings without needing an external API, and is

optimized for datasets under ~100k entries ¹⁹ ²¹ – which is likely sufficient for our knowledge base size. For instance, using a MiniLM 384-dimensional model, we can embed each document and store those vectors. This gives us semantic search capabilities on-premise.

- **Vector Store:** We will need a place to store and query the vectors. Options include using an open-source vector database like **Qdrant** or **Weaviate** (both have free versions). Qdrant, for example, can be run via Docker and supports similarity search with filters. Qdrant's cloud free tier allows ~1 million vectors of dimension 768 ²², and Pinecone (another managed vector DB) allows ~300k vectors at 1536 dims on its free plan ²³. We could self-host Qdrant on the same server as the Go app (if resources allow) or use Qdrant's free cloud. Given our desire to stick to free tools, running Qdrant locally or using its free tier is ideal. We'll integrate the Go backend with this vector store (Qdrant has a REST API and also a Go client SDK).

- **Similarity Search:** Whenever needed (e.g. an admin runs a “find duplicates” action, or we nightly check for similar docs), we compute the embedding of a given document and query the vector store for nearest neighbors (using cosine similarity or dot product). If a high similarity is found above a certain threshold, we flag those docs as potential duplicates/conflicts. This can aid the conflict detection process by narrowing down candidates that a human or an LLM can analyze.

- **Conflict Detection via LLM:** To determine if two similar documents truly conflict (i.e., have contradictory statements), we can employ an LLM to analyze them. An open-source LLM (like GPT4All, LLaMA 2, or smaller fine-tuned models) might be used, but these may be less reliable for nuanced contradiction detection. If high accuracy is needed, we might call on a stronger model via API (see Azure below). However, we could attempt a local approach: e.g., use an LLM prompt that reads both documents and asks “Are there any conflicting instructions or facts between Doc A and Doc B?” If we have a reasonably capable model hosted (like LLaMA-2-7B with fine-tuning), it could answer. Research indicates LLMs can detect inconsistent claims to a good extent – for instance, experiments with GPT-4 have shown around **90% recall in detecting inconsistent claim pairs in clinical evidence** ²⁴. This suggests our approach of using LLMs for conflict checking is feasible. We could integrate a smaller model for on-prem (if available) or use Azure for the heavy analysis.

Azure RAG Approach: Using Azure's cloud AI services is another path, especially if we need more powerful language understanding:

- **Azure Cognitive Search** could serve as our vector store and search solution. Azure provides a Free tier for Cognitive Search that supports up to 50 MB of indexed data (which is enough for a few thousand documents) ²⁵. We can index the knowledge base documents there, and use Azure's semantic search or vector search capability (they offer semantic ranking and even built-in vector search with the new cognitive search features). This service can return similar documents for a query or detect duplicates by clustering. The free tier allows one free search service per subscription and is a cost-free way to experiment ²⁵. Additionally, Azure's “agentic retrieval” (which is part of their RAG pipeline) offers the first **50M tokens per month free** for processing queries ²⁶, which is quite generous – likely far above our needs.

- **Azure OpenAI (LLM) for Q&A and Analysis:** We could use Azure's OpenAI service to run an LLM (like GPT-3.5 or GPT-4) on our data. In a typical RAG pipeline, we'd use cognitive search to retrieve relevant doc chunks, then prompt the LLM to answer questions or summarize differences. For conflict detection, we might feed two documents into GPT and ask if they contradict each other. Azure OpenAI has a cost, but for planning purposes we can note it's *relatively low per text volume*. For example, the state-of-the-art embedding model (text-embedding-ada or the new text-embedding-3) costs on the order of **\$0.02 per 1,000,000 tokens** for the small model ²⁷, which is extremely cheap (0.00002 per 1K tokens). Generating a few embeddings for our docs would cost fractions of a cent. The question-answering via GPT-3.5 is about \$0.002 per 1K tokens. Given these rates, even a few hundred conflict checks per month would likely stay well within a free-tier credit or minimal expense (a few dollars). Azure might also provide some free credits initially. We will present the option to the project stakeholders: an Azure-

based solution would involve cost if scaled, but it might be easier to implement (leveraging Azure's managed infrastructure and powerful models).

Recommendation: We aim to primarily implement the open-source solution to avoid external dependencies and costs. Specifically, integrate a **Go-based vector search** (via Qdrant or similar) and use an **open-source embedding model** to enable semantic similarity features. This empowers features like: a **semantic search bar** for the knowledge base (users can search by meaning, not just keywords), automatic suggestion of related articles, and detection of duplicates. For conflict detection, we might start with a simpler approach: flag high-similarity docs and provide them to a human admin to review for conflicts. If time permits, integrate an LLM to automatically highlight contradictory sentences. Azure RAG can be an optional extension or backup if the local approach proves insufficient – we'll document how to plug in Azure Cognitive Search or OpenAI if needed, including cost estimates (e.g. "Using Azure's embedding and GPT-4 to compare two documents of ~1000 tokens each might cost ~\$0.03 per comparison – which for occasional use is negligible, but not zero").

By **embedding the documents and using vector similarity**, we fulfill the requirement for document similarity detection. By optionally employing LLM reasoning on top, we tackle the conflict detection in a smart way. This combination of vector search and LLM reasoning is essentially what RAG (Retrieval-Augmented Generation) entails – either done fully in-house or with Azure's help.

Finally, to manage vector data for **free-tier hosting**: if using Qdrant, we can run it on a free cloud instance; if using Pinecone, the free plan (up to ~300k vectors, one pod) is sufficient for dev/test ²³; if using Azure, the free search covers our needs ²⁵. We'll ensure whichever vector store, it's deployed with encryption and proper access (Qdrant supports API keys, and even has JWT/RBAC security for its cloud ²⁸ ²⁹). The vector index can be periodically rebuilt as documents update (freshness of embeddings can be handled by re-embedding a doc when it's edited).

In summary, the AI integration will make the platform **intelligent** – automatically relating content and catching contradictions – using **open tools (LLMs and vector DBs)** or **Azure's services**, both of which are viable with minimal or no cost at low scale ²² ²⁶. This adds significant value to the knowledge base component beyond a traditional static documentation system.

Architecture & Tech Stack

High-Level Architecture

The system follows a typical **three-tier architecture** with a RESTful API backend, a web frontend, and a database (plus auxiliary services for search and messaging). Key components and their interaction:

- **Angular Frontend:** The client is a Single Page Application (SPA) built in Angular (we'll use a modern Angular version). The Angular app communicates with the backend via HTTPS AJAX calls (using Angular's HttpClient). It also manages real-time channels (WebSockets) for chat. The app is structured with modules for different portals (e.g. Admin module, Doctor module, Patient module) for organization and lazy loading. The UI uses a responsive design (possibly Angular Material components) given that doctors or patients might access it on various devices. We will use routing guards to protect pages based on role (e.g. prevent a patient from accessing admin URLs). Angular will handle initial authentication (storing JWT or session info) and provide a rich user experience with the dashboards, forms, and charts described. Once built, the Angular app will be deployed as static files (HTML/JS/CSS).

- **Go Backend (REST API):** The backend is a Go server that exposes RESTful endpoints for all core functionality. We might structure it with a modern Go web framework for productivity – popular choices include **Gin** or **Echo** (for fast HTTP routing, middleware support) or **Fiber** (Express-like). For example, using Gin, we can define routes like:

`GET /api/patients/:id` – returns patient info (allowed if auth user is that patient or their doctor or admin).

`POST /api/prescriptions` – doctor creates a new prescription (with JSON body including patient, medicine, etc.).

Similarly, endpoints for documents, messages, login, etc., will be set up. We will implement middleware for auth (JWT verification and role checking for each route). Logging middleware will record accesses for audit.

The Go backend will implement business logic in a structured way (perhaps following a layered architecture: routers -> handlers -> service layer -> data access). This separation makes it maintainable.

- **Database (SQL):** A relational database (PostgreSQL or MySQL) stores the persistent data. We prefer **PostgreSQL** for its robustness and features (like JSON support, full-text search, and available extensions like pgvector if needed). But MySQL could also suffice. We will design a relational schema with tables for Users, Patients, Doctors, Appointments, Prescriptions, MedicalRecords (or Visits), Messages, KnowledgeDocs, DocVersions, etc. (An example schema is discussed in the next section). We can use an ORM in Go like **GORM** for quicker development or use low-level database/sql with query building. Given the need for performance and straightforward queries, using an ORM for basic CRUD and writing raw SQL for complex analytics might be a good balance. We will ensure to add appropriate indexes (e.g. on foreign keys like patient_id, on document titles for search, etc.).
- **Vector Search Service:** If we integrate a separate vector database (like Qdrant or Pinecone), this acts as an auxiliary component. The backend will communicate with it when performing similarity queries or updating embeddings. For an open-source self-host, we might run Qdrant as a container alongside the Go app. The architecture then has an extra service: **Go -> Qdrant (via REST gRPC)** for similarity search tasks. If using Azure Cognitive Search instead, the Go backend would call the Azure Search REST API. In either case, this component is logically separate from the main SQL DB, optimized for semantic queries.
- **Background Job Scheduler:** Within the backend, we will have background routines for tasks like sending reminders, running nightly scans, etc. We can implement this as part of the Go process (using goroutines and timers or a cron library). For example, a goroutine could tick every minute to check if any reminder needs sending at that time. Alternatively, we could deploy a separate Go worker process to handle queued tasks – but given modest load, a single binary can handle both API requests and background jobs concurrently. We will just need to ensure thread-safety if both try to access certain resources.
- **Hosting & Deployment:** All components will be containerized using Docker for consistency (a Dockerfile for the Go API, another for Angular build if needed, or we can serve Angular static from Go or a CDN). For a free-tier deployment:
- The **Angular app** can be hosted on a static site hosting provider for free. For instance, **Netlify** offers free hosting for static websites with generous bandwidth ³⁰. We simply upload the `dist/` build and Netlify will serve it over CDN with HTTPS. Alternatives: GitHub Pages (if repository is public), Vercel, or Cloudflare Pages – all have free plans for static frontends.

- The **Go API** and **Database** can be hosted on a free-tier cloud service. A good choice is **Render.com** which provides free plans for web services and even a free PostgreSQL database ³¹. We can deploy the Go API on Render's free web service (which can run continuously with certain limitations, e.g. it might sleep on inactivity). Render also offers a free Postgres instance (100 MB storage). Another option is **Railway.app** with \$5 free credit monthly which can cover a small container and DB. **Fly.io** could host a small Go app on their free tier as well (they have a few monthly free VM hours). If more control is needed, we could use a small VM on AWS/GCP's free tier or Oracle Cloud's Always Free (Oracle gives 2 small VMs and a free DB that could be used). For simplicity, using a PaaS like Render or Railway is ideal – push code and it deploys. According to free-for.dev, Render's free plan includes SSL, auto deploys, and private networks for the DB ³¹, making it fitting for our needs.
- The **Vector store** can be hosted similarly: if using Qdrant, we could deploy it in a Docker on the same free VM as the Go server (ensuring the VM has enough RAM for Qdrant which uses Rust – a small dataset should be fine). If using Pinecone's free cloud, that's an external service but free for our size. If using Azure free cognitive search, that's also an external cloud component but free. So hosting cost is not an issue for this part.
- **Domain & SSL:** The free hosts typically provide HTTPS on their domain (e.g. myapp.onrender.com). That's sufficient for development. Eventually, one could map a custom domain (which on some services might require a paid plan or at least the domain purchase). But for now, we assume default hostnames with SSL are fine.
- **Scalability:** The chosen stack (Go with a SQL DB) can scale vertically on free tiers to an extent, but not infinitely. However, the design allows easy migration to a more scalable setup if needed – e.g. containerize and deploy to a cloud cluster. The stateless Go API can be replicated behind a load balancer, the DB can be upgraded to larger instance, etc. The architecture is modular: we could even separate some services (like have a dedicated microservice for the knowledge base or for sending notifications) if the team decides to, but initially we will implement it as one service for simplicity.

In diagram form (conceptually):

Frontend (Angular SPA) – (HTTPS REST + WebSocket) – **Go API Server** – (SQL queries) – **PostgreSQL/MySQL DB**; and the Go API also connects to **Vector DB (Qdrant)** for semantic search; Go API uses **SMTP/SMS gateways** to send emails/SMS; Users (Doctors/Patients/Admins) access via their browsers to the Angular app.

(If an architecture diagram image were to be included, it would show Angular in the client, Go API in server, DB, vector service, and external email/SMS services all connected.)

Data Model & Relationships

We design the database with a clear schema separating different domains (medical records vs knowledge base). Below is a simplified entity-relationship outline for core tables (with relationships in parentheses and key fields):

- **User** – stores common user account info. Fields: `user_id (PK)`, `name`, `email`, `password_hash`, `role` (e.g. "doctor", "patient", "admin"), etc.
- **Relationship:** A User may have a Doctor profile or Patient profile. We could use a single table for all users with role discriminators, or separate tables. To allow a person to potentially be both

(edge case), separate might be better. But typically, doctors and patients are distinct populations, so we could simply have one table and differentiate by role.

- **Doctor** – stores doctor-specific info. Fields: `doctor_id` (PK, also FK to `User.user_id`), `specialization`, `department`, `license_number`, etc. One-to-one with a `User` entry (when `User.role = doctor`).

- *Relationship*: Doctor has many Patients (through `Patient` records that list their primary doctor or through encounters). Possibly a Doctor-Patient linking table if many doctors per patient or vice versa (for group practices or referrals). But assume each patient has one primary doctor for now, stored in `Patient`.

- **Patient** – stores patient-specific info. Fields: `patient_id` (PK, FK to `User.user_id`), `date_of_birth`, `gender`, `contact_info`, `primary_doctor_id` (FK to `Doctor`), etc. Also, insurance details if needed.

- *Relationship*: A Patient is linked to one primary Doctor (Doctor has many Patients). Patient has many `MedicalRecords`, `Prescriptions`, etc.

- **MedicalRecord / Encounter** – represents a clinic visit or entry in the patient's record. Fields: `record_id` (PK), `patient_id` (FK), `doctor_id` (FK), `date`, `notes` (text), `diagnosis` (could be text or code), etc. We might also break out **Diagnosis** and **TreatmentPlan** tables for more structured data, but a simplified model could have them in the record or in a JSON field.

- *Relationship*: Many records per Patient; each record linked to the Doctor who wrote it.

- **Prescription** – a prescribed medication entry. Fields: `prescription_id` (PK), `patient_id` (FK), `doctor_id` (FK), `date_prescribed`, `medication_name`, `dosage`, `frequency`, `duration`, `instructions` (text). If integrating with pharmacies, might have pharmacy info or status.

- *Relationship*: Patient can have many prescriptions; each prescription has the prescribing Doctor. We might link a Prescription to a specific `MedicalRecord` (encounter) via a foreign key `record_id` if prescriptions are usually given during a visit – that would make sense to tie them to the context.

- **TestResult / Attachment** – stores files or results uploaded. Fields: `file_id` (PK), `patient_id` (FK), `uploaded_by` (FK User who uploaded, could be patient or doctor), `date_uploaded`, `description`, `file_path` or blob. If it's a structured lab result, we might have `result_type`, `value`, etc., but often they come as PDF or image.

- *Relationship*: Belongs to a Patient. Could also have an optional `record_id` if it was part of a specific encounter, or null if patient uploaded independently.

- **Message** – for chat/messages. Fields: `message_id` (PK), `sender_id` (FK User), `receiver_id` (FK User) (for direct messages), `content` (text), `timestamp`, `is_read`. For group or thread contexts, we might have a `thread_id` or if attaching to record/doc, a `context_type` and `context_id`. Possibly we have separate tables: one for

direct messages (doctor-patient) and one for comments on documents. But we can unify by including context fields.

- *Relationship*: Each message has sender and receiver users. If it's a comment on a knowledge doc, we set receiver maybe as null and use context fields. Alternatively, separate table **DocComment**(doc_id, user_id, comment, timestamp). Similarly, **RecordComment**. For simplicity, treat messaging and comments somewhat separately: direct Message table for user-user chat; and Comment tables for entity-specific discussions.

- **KnowledgeDoc** – represents a knowledge base document (e.g. an article or guideline). Fields: doc_id (PK), title, content (text or HTML), author_id (FK User), created_date, updated_date, health_score, status (active/archived). Might also have metadata like category/tags.

- *Relationship*: Could have many Versions and many Comments. Possibly link to other docs if needed (like references).

- **DocVersion** – stores historical versions of KnowledgeDoc. Fields: version_id (PK), doc_id (FK), version_number, content_snapshot, timestamp, editor_id (FK User). This allows us to retrieve older content if needed or show diffs. Alternatively, use a versioning system or git backend, but that's complex; a simple table works.

- **KnowledgeAlert / ConflictFlag** – an optional table to track flagged issues in docs. Fields: alert_id, doc_id, type (e.g. "stale" or "conflict"), details (text describing the issue, e.g. "Similar to doc #5 on dosing"), flagged_date, resolved_date. This helps admins manage what needs attention.

Other tables could include **Appointment** (with date, patient_id, doctor_id, status), **Notification** (if we log each reminder sent or pending), etc., but the above covers main entities.

Relationships summary: A Doctor has many Patients; a Patient has many MedicalRecords and Prescriptions; a Doctor has many MedicalRecords (for different patients) and many Prescriptions they wrote. KnowledgeDocs are independent of patients but authored by users (usually doctors or admins). We ensure foreign keys with ON DELETE actions appropriately (e.g. if a user is deleted – which might not happen often – we either cascade or better mark inactive to preserve data integrity).

To illustrate, consider an example data scenario: Dr. Alice (User 1, role=doctor) has Patient Bob (User 2, role=patient). Bob has two MedicalRecords: a check-up on Jan 10 (record 101 by Alice) and a follow-up on Feb 20 (record 102 by Alice). Bob has one Prescription (prescription 201 for Blood Pressure medication, tied to record 101). Bob uploaded a lab result file (file 301) which is linked to record 102. Meanwhile, the knowledge base has a document "Managing Hypertension" (doc 1001) authored by Alice. That doc has version 1 and 2 in DocVersion table (after an update). The system computed a health_score for it (maybe 95%). A second doc "Blood Pressure Guidelines" (doc 1002) exists, and the similarity system flagged a conflict with doc 1001 – an entry in KnowledgeAlert was made referencing both. Dr. Alice sees that alert, compares docs, and updates doc 1001 to resolve the discrepancy.

The schema will be implemented via migrations (using a tool like golang-migrate or GORM's automigrate if using GORM). We will ensure to create **indexes** on important fields: e.g. index on Message (receiver_id) for quick retrieval of your inbox, index on KnowledgeDoc (title) for quick search, etc.

Additionally, we might use some advanced DB features: **pgVector** extension if we wanted to store embeddings in Postgres itself and use vector similarity directly in SQL (this is an alternative to a separate vector DB). Postgres pgvector allows storing vector columns and a `<->` operator for similarity search. If we go that route, we could avoid a separate Qdrant service by storing document embeddings in a column of KnowledgeDoc and indexing it (pgvector supports approximate search indexing). In fact, a tutorial shows building semantic search with Postgres + pgvector ³² ³³. This could simplify the stack further. The decision will depend on what's easiest on the chosen free host (some managed Postgres like Supabase come with pgvector enabled).

In any case, the data model is designed to capture all necessary information while maintaining normalization and referential integrity. We will use **transactions** where needed (e.g. creating a prescription and linking it to a record). The design also facilitates analytics – e.g. we can query how many prescriptions per doctor easily via SQL grouping.

(If needed, an ER diagram image could be provided; but since this is a text format, the above description and relationships suffice.)

Open-Source Libraries & Tools

We will utilize several open-source libraries to accelerate development and implement features efficiently:

- **Go Web Framework:** Likely **Gin Gonic** (MIT licensed) for its speed and minimalist syntax. It will handle routing and middleware (auth checks, logging). Alternatively, **Echo** or **Fiber** can do similarly. Example: using Gin's JWT middleware to secure routes. This avoids writing boilerplate HTTP handling from scratch.
- **ORM/Database: GORM** (for Go) is a popular choice to interact with the SQL database in an object-oriented way. It supports both Postgres and MySQL. We can define Go structs for each table and let GORM handle SQL generation. For complex queries (like joining for dashboards, or custom analytic queries), we might use the database's JSON or window functions via raw SQL. GORM is open-source (MIT). If not GORM, we might use the lower-level but lighter **sqlx** for easy query-to-struct mapping.
- **Authentication:** For JWT, the **golang-jwt** library (successor of dgrijalva's jwt) can be used to sign/verify tokens. For password hashing, **bcrypt** from Go's x/crypto or **golang.org/x/crypto/bcrypt** will be used. These are well-tested components.
- **Scheduling/Crons:** The **robfig/cron** library (Apache 2.0) for Go allows scheduling jobs with a cron syntax. We can use it to schedule nightly tasks or periodic checks easily instead of writing our own timer logic.
- **Email/SMS Integration:** Use official client libraries if available. Twilio provides a Go helper library for sending SMS. SendGrid has a Go library for sending emails. These save us from crafting raw HTTP requests. Both are free to use (and just require API keys).
- **Charts and Frontend UI:** On Angular, we will likely use **Angular Material** for UI components (it's Google's open-source UI kit for Angular, under MIT license). It provides ready-made components for forms, navigation, tables, modals – accelerating UI development with a consistent look. For charts, **ng2-charts** (which is an Angular wrapper around Chart.js) is a simple way to embed charts ³⁰. Chart.js is MIT licensed. NGX-Charts (based on D3) is another option if we want more complex charts – it's also open-source (Apache). For calendar UI (doctor's schedule), we can use a library like **Angular Calendar** (e.g. angular-calendar library) to display events in a calendar format. There are open-source components for this as well.

- **Text Editor for Knowledge Base:** If we want rich text editing for knowledge articles, we can integrate an open-source WYSIWYG editor in Angular, like **TinyMCE** (which has an open core) or **Quill.js** (free) to allow admins to format documents with headings, lists, etc. The content can be stored as HTML in the DB. Alternatively, use Markdown and an editor for it.
- **Full-Text Search:** If basic keyword search is needed (aside from semantic), Postgres's full-text search can be used, or an open-source search engine like **Elasticsearch** (but that's heavy for free tier). Likely we'll lean on Postgres or the vector search for search functionality.
- **AI/ML Libraries:** For the open-source embedding approach, we mentioned the **kelindar/search** Go library that can generate embeddings using local models ¹⁹ ²⁰ . We will use that to avoid external dependency. If not that, another route is using Python's **sentence-transformers** in a microservice. But given the interest in keeping everything in Go, kelindar's approach is promising. It even supports GPU if available, but we can run CPU for small scale. We will download a small model in GGUF format (the repo's example uses MiniLM-L6-v2.Q8_0, which is a 384-dim model quantized 8-bit – very fast and small ³⁴).
- **RBAC Library:** We may integrate **Casbin** (which has a Go library under Apache license) to define and enforce roles and permissions. Casbin would allow us to externalize the policy, for example: define roles "doctor" can access `GET/POST /patients/:id/records` only if id matches, etc., possibly using attributes in requests. Casbin supports hierarchical roles (RBAC1) so we can define admin > doctor > patient role inheritance ¹⁴ . If Casbin feels too heavy, a simpler custom middleware might suffice (a series of `if role == ... and resource ...` checks), but Casbin gives flexibility if requirements grow (like adding a nurse role later).
- **Encryption:** Go's standard library covers most needs (crypto/aes, crypto/rand for generating keys, etc.). For any advanced needs, libsodium's Go bindings or x/crypto can be used. We might use an existing package for .env configuration to supply encryption keys.
- **Testing and Validation:** For input validation, a library like **go-playground/validator** helps validate struct fields (e.g. ensuring emails are valid format, required fields present). This is open-source and commonly used. For testing, we'll use Go's built-in testing framework, and perhaps tools like **Testify** for assertions to speed writing tests.

All these tools are free and open-source, aligning with the requirement of not using proprietary paid components. We avoid any commercial libraries. Even the AI models and libraries chosen are open (MiniLM is originally by Microsoft but released, and implementations to use it like sentence-transformers or Kelindar's library are open-source).

Using these libraries accelerates development while keeping costs at zero. We will document in the project README which libraries are used for what, to ensure compliance and ease of updates.

Deployment & Hosting (Free Tier)

We target a zero-cost deployment using free-tier services:

- **Go API Hosting:** We recommend **Render** or **Railway** for hosting the Go backend. Both offer a free plan that can run a small web service. For example, Render's free web service can run a Go server with 512 MB RAM and shared CPU, which is sufficient for a prototype. It auto-deploys from GitHub and provides HTTPS ³¹ . The only limitation is it may sleep after 15 minutes of inactivity (on free tier), adding a few seconds delay on first request to wake – acceptable in early stages. Railway similarly can run the container and likely won't sleep as long as usage is within the \$5 credit. Another alternative is **Fly.io**, which allows small VMs near globally – their free tier includes 3 tiny VMs (256MB) and some outbound bandwidth. Fly could be used to host the Go API as a Docker container, and even the Postgres in another (they have volumes etc.). We will

consider ease-of-setup: Render might be simplest (just `render.yaml` config and push code). We'll containerize the Go app anyway, so moving to any platform is easy.

- **Database Hosting:** If we use Render, we can add a managed PostgreSQL instance on free tier (they have one free instance with 100MB). 100MB can actually handle quite a lot of text data (thousands of records and docs). If we need more, Railway's free credit could host a larger Postgres. **Supabase** is another interesting option: they provide a free Postgres database (up to 500MB) plus they have pgvector enabled (and a UI, etc.). Supabase free tier might be very useful if we want vector search directly in Postgres, plus it has an API (but we'd still use our Go backend, we can just treat it as a hosted DB). For MySQL, there's **PlanetScale** free tier (but it's more suited to serverless usage and doesn't allow certain transactions easily). Given our leaning to Postgres and possibly needing extensions, Supabase or Render is preferable. Supabase's always-free tier includes 2 CPU and 1GB RAM for the DB, which is generous, and 500MB storage, plus 50MB outbound (for sending results) which is okay. So we might choose Supabase for DB and still host Go on Render. The Go app would then connect to Supabase's connection string (which is a remote Postgres). The latency is fine if in same region. This decoupling might be fine since both are free.
- **Angular Frontend Hosting:** For the Angular app, **Netlify** is a great choice – simply drag-drop the build or link Git for CI. Netlify's free tier offers 300 build minutes and 30GB/month bandwidth ³⁰, more than enough for us. It also auto-deploys on commits. It provides a `<your-app>.netlify.app` domain with SSL. Similarly, **Vercel** (mostly for React/Next, but can host any static) could be used; or GitHub Pages if we keep it straightforward (though GH Pages needs the repository public unless using Actions). We'll likely go with Netlify for ease. Another free host is **Cloudflare Pages** – also great for static and has global CDN.
- **Domain Name:** If a custom domain is desired, we can use a free subdomain for now (like netlify's subdomain or render's). If needed, one could use a free `.tk` domain or similar just for demonstration, but that's optional and not asked.
- **Vector Store Hosting:** If using Qdrant, one approach: run Qdrant on the same Render app instance (maybe not possible on their managed web service). Alternatively, host Qdrant on **Deta Space** or **Railway** (Railway could run a Docker of Qdrant from its image, using some of that free credit). Qdrant also has a cloud – it's not completely free unlimited, but they mentioned a free tier for ~1M vectors ²². We could sign up for that and use their cloud endpoint (they allow 1 replica, limited performance but fine). That would be convenient since they manage it and provide an API key. Pinecone's free tier could also be used similarly by calling its API (limited to one index, 1 pod, ~300k vectors which is fine ²³). If we do use external (Qdrant cloud or Pinecone), it's free but not self-hosted – however the prompt only said free-tier, which those are. Using external might simplify not consuming our server resources. We should note that Qdrant Cloud free requires a credit card (for verification) but doesn't charge if usage is within free limits. For a fully open route, if deploying on a VM (like Fly or Oracle free VM), we could Docker compose the Go app and Qdrant together. For initial simplicity, using Pinecone or Qdrant Cloud is fine – minimal integration needed (just HTTP calls from Go).
- **Scaling on Free Tiers:** Free tiers often have limits on CPU/memory. We should test that our app with typical usage (a few concurrent users) runs within those constraints. Go is efficient, and our use of external services for heavy tasks (like vector search on its own service, or sending email via external API) means the Go container isn't overloaded. We should use caching where possible (maybe cache frequently read docs in memory to reduce DB hits, which is safe on a single-

instance scenario). If user base grows, we'd move to a small paid plan or optimize further. But given the timeline (April 22), free tier is fine for prototype or initial launch.

- **Continuous Integration/Deployment:** We can set up GitHub Actions (free) to run tests on push and even build the Docker images. Then Deploy to Render or Netlify via their integrations. This ensures a smooth dev workflow.

In summary, a possible deployment configuration: - Frontend: Netlify (free) – delivering the Angular SPA globally via CDN ³⁰ . - Backend API: Render free web service (HTTPS endpoint with Go running) ³¹ . - Database: Supabase free Postgres (500MB) or Render free Postgres (100MB) – accessible from the Go API (with proper network settings or public connection string). - Vector DB: Qdrant Cloud free (1M vectors) or as a service in same environment. - Email/SMS: Twilio trial for SMS, SendGrid free for email – the Go backend just calls these external services as needed (no hosting required for them, just accounts configuration). - Monitoring: We can rely on these platform's dashboards. If needed, use something like Sentry's free tier for error tracking in frontend and backend, but that's optional.

This setup ensures **zero monthly cost** while covering all components. All chosen providers have an acceptable free usage for our scale. (We will be mindful of any free-tier conditions, like sleeping dynos on Render, to mitigate user impact – e.g., perhaps ping the API periodically to keep it warm, which some people do using free cron services.)

By following this deployment plan, we fulfill the requirement of using **only free-tier services** for hosting the Go server, Angular app, SQL database, and vector store. We will document how to set up each (e.g. environment variables on Render for DB URL, etc.) so that anyone can replicate the deployment without incurring cost.

Implementation Plan (Sprint-wise to April 22)

We propose an agile implementation in **sprints**, ending by April 22. Assuming roughly ~10 weeks available, we can have five 2-week sprints (or adjust as needed). Each sprint will produce a set of features (with priority on core functionality first, then enhancements):

- **Sprint 1: Project Setup & Core Auth (Weeks 1-2)**

Goals: Establish the project foundation and basic authentication.

- Set up the GitHub repository and initialize Go module and Angular workspace. Implement CI (GitHub Actions) for test/build.
- Define the initial database schema and implement migrations. Set up a local Postgres for dev.
- Implement the **User model and Auth APIs** in Go. This includes user registration (if patients self-register or admin adds users) and login endpoint issuing JWT. Secure password storage (bcrypt) is done.
- Basic RBAC: enforce role claims in JWT (include role in token) and write middleware to check protected routes. At least differentiate patient vs doctor vs admin access in a simple way.
- Angular: Create a login page and registration page. Use Angular forms to capture credentials. Implement auth service to call login API, store JWT (likely in localStorage for now, with TODO to secure it, or use HttpOnly cookie mode if decided). After login, redirect user to appropriate dashboard route based on role.
- Set up initial Angular routing with placeholders for Doctor Dashboard, Patient Dashboard, Admin Dashboard components (just simple welcome text for now). Protect routes using route guards that check role in a client-side token (note: still rely on server for real security).

- **Verification:** By end of Sprint 1, a user can register (or an initial admin user is inserted manually), then log in and see a basic dashboard page. We have the skeleton in place to expand features.
- **Deliverables:** Basic working auth (tested via unit tests in Go for password and JWT logic, and integration test hitting login), and project scaffold on both ends. Possibly deploy this minimal app on the chosen free hosts to ensure pipeline works (Netlify deploy of the static app, Render deploy of backend).

• **Sprint 2: Knowledge Base Module & Document Monitoring (Weeks 3-4)**

Goals: Implement the Knowledge Base management and health monitoring features.

- Create KnowledgeDoc CRUD: Go endpoints for creating, retrieving, updating, deleting knowledge base documents (admin/doctor role allowed). Also endpoints to get lists of docs, maybe filter by status. Implement versioning: when updating, create a DocVersion entry (or use a soft-delete approach for now).
- Compute and store initial **health score** for docs. For start, health_score could be a simple function of time since last update (e.g. 100% if updated <1 month, down to 0% if >1 year). We'll refine with more factors later. Implement a scheduled job or on-read calculation to flag stale documents (set a boolean or create entries in an Alert table if beyond threshold). For example, a nightly job: mark any doc older than 6 months as stale.
- Set up the knowledge base dashboard for admin: an API that returns summary metrics (# docs, # stale, maybe list of stale ones, etc.). Also an API for conflict alerts (initially, we may simulate one or two manually to test UI).
- Implement **document search** endpoints: initially just by keyword (SQL LIKE on title/content). Later we'll replace/enhance with vector search. But have something in place so the knowledge base is navigable.
- Angular: Build Knowledge Base UI for admins. A page listing all documents (title, last updated, health score, flags). Use a data table with sorting/filtering. Highlight stale ones (e.g. red text if health_score < some value).
- Document edit page: Provide a form or rich text editor to edit content. We can integrate a basic editor (maybe just a `<textarea>` first, or a markdown editor). Include a field for version notes if needed. On save, call update API. Show version history (list versions with date, perhaps allow revert by copying content from old version).
- Display conflict alerts: On the KB list, if a doc has a conflict flag, show an icon. Admin can click to see details (e.g. "Doc X and Doc Y conflict on recommended dosage of DrugZ"). For now, since we might not have AI fully integrated yet, we can manually trigger one for demonstration or use a placeholder logic (like if two docs have very similar titles).
- **Document Similarity (Phase 1):** Integrate the vector embedding library on a basic level. Possibly in this sprint or next – but at least decide and set up Qdrant or pgvector. Maybe as a stretch goal in Sprint 2: get all documents embedded (could do using a dummy small model or even call an API like OpenAI's embedding for quick result, just to have data), and implement an API to find similar documents to a given one. This could be tested with a few sample docs. The heavy LLM analysis part can wait.
- At end of Sprint 2, the knowledge base functionality should be usable: Admin can add docs, edit them, see stale flags, and get basic suggestions of duplicates. This meets the original KB monitor's main points (document CRUD, versioning, freshness tracking, some conflict detection mechanism).
- **Deliverables:** Knowledge base management UI and APIs working, basic scoring and alerts visible.

• Sprint 3: Doctor & Patient Records Module (Weeks 5-6)

Goals: Implement the EMR features: patient records, prescriptions, file uploads, and doctor/patient views.

- Extend the schema and Go models for Patient, Doctor, MedicalRecord, Prescription, etc.

Implement endpoints for:

- Doctors to **create and update medical records** (e.g. `POST /patients/{id}/records` to add a visit note for patient {id}, only accessible to that patient's doctor).
- Doctors to **create prescriptions** (`POST /patients/{id}/prescriptions` or as part of record endpoint).
- Patients to **view their records** (`GET /me/records`) and **view prescriptions** (`GET /me/prescriptions`). These should contain necessary details. Possibly format dates nicely or calculate if a prescription is currently active.
- File upload: an endpoint for uploading a file (lab result). We will likely use a multipart form handler in Go. The file can be stored on disk or in the DB as blob. For simplicity and given small scale, storing on disk (or cloud storage) is fine; we just store a path/URL in the DB. But since we want fully free, maybe we store in DB to avoid needing an S3 (though some S3 like Cloudflare R2 have free tier). Alternatively, encode file as base64 in JSON (not great for large files). We'll likely configure the API to accept an upload and save it in a `uploads/` folder on the server (ensuring the server can serve it securely – or requiring authentication to download, possibly via another endpoint). This might require our host (Render) to have persistent disk – Render does offer persistent disk on paid plans, but not sure on free. If not, an alternative is to use something like Cloudinary free or S3 free, but that complicates. Perhaps easier: store small files in the DB as bytea (Postgres). We'll do that for now given low volume.
- Implement listing of files for a patient and retrieval (download endpoint that streams the file bytes). Ensure that only the patient or their doctor can access a given file.
- Implement **Medication reminders logic** (back-end only this sprint): When a new prescription is created with a frequency schedule, register it in the scheduler. For now, we might just log that "Would send reminder at X" if actual sending integration isn't done yet (or we send emails to a test address). But get the scheduling logic correct. Use cron library or a simpler time.After loop for each (but that gets tricky if many). Possibly insert reminders into a "PendingReminders" table with timestamp, and have a background goroutine that checks every minute for any due reminders. That's a robust approach. We'll flesh out the actual send in next sprint.
- Angular Doctor UI:
 - A page to view **patient list** (doctors should see a list of their patients). API: `GET /doctor/me/patients` returns basic info of each. The doctor selects a patient to manage.
 - Patient detail view for doctor: shows patient demographics, and tabs or sections for "Records/Notes", "Prescriptions", "Uploads/Labs". The doctor can create a new record (display a form with fields for note and diagnosis, etc.), and a new prescription (fields for med name, etc.). Use reactive forms. When submitted, call the appropriate API and update the view.
 - The doctor should be able to download files the patient uploaded (just a link that calls our download API). Possibly preview if image/PDF – but for simplicity, download link or embed PDF if we can.
 - Maybe show a simple visual indicator if patient has something overdue (like if last visit was long ago – but that's more of an alert, can leave if time).
 - Doctor's main dashboard: list of upcoming appointments (if we have scheduling), and recent notifications. We might integrate scheduling next sprint if not done yet. At least have a placeholder "Appointments" page this sprint if not fully functional.

- Angular Patient UI:
 - A “My Health” page showing their info. They can view a list of their visit records (with dates and doctor notes). They can click to see details of each note (maybe toggling the full text).
 - A list of current prescriptions (with name, dosage, etc. and maybe a visual indicator if active or completed). Possibly allow patient to mark a medication as taken each day (if we implement tracking, could skip for now).
 - An interface to upload a file (lab result): e.g. a form with file input and description field. They submit, Angular sends multipart form to API. After success, display in their “My Files” list.
 - Possibly show upcoming appointments (or a message “Contact office to schedule” if scheduling not yet done).
 - Patient should also have access to the knowledge base articles if we want (maybe a public health education section?). This wasn’t explicitly asked, but could be value-add – though if the KB is internal, skip.
- All these UI pages will be behind auth guard, and we ensure to hide features not applicable (e.g. patient doesn’t see admin menu). Use Angular’s **Route Guards** to protect routes by role, and Angular *CanActivate* logic to redirect if not allowed. This duplicates server checks but improves UX by not even showing links.
- Testing: Write unit tests for the Go handlers (especially ensuring RBAC logic works – e.g. patient can’t get someone else’s data, etc. simulate with JWTs). Also test critical functions like medication schedule generation. For Angular, run `ng test` and ensure basic component creation and maybe a service test for the patient service.
- **Deliverables:** By end of Sprint 3, doctors can manage patient records and patients can view their data via the UI. The core EMR functionality is in place, albeit maybe not as polished.
- **Sprint 4: Communication, Reminders & Refinements (Weeks 7-8)**

Goals: Add the chat messaging feature, finalize the email/SMS reminder sending, and implement any scheduling or calendar features. Also integrate any needed external services (Twilio, etc.) now.
- **Chat System:** Implement WebSocket endpoint in Go for messaging. Use Gorilla WebSocket. Clients will connect to, say, `/ws` with their auth token. The server maintains a map of connected users. Implement basic broadcast to target user. Develop a protocol (maybe simple: JSON messages with fields: to, from, content). Also save messages to DB upon receive.
 - Angular: Create a service for WebSocket connection. In the patient view, add a “Messages” section where patient can send message to doctor (assuming one primary doctor relationship). Or a generic “Contact Doctor” button that opens a chat dialog. The Angular component will connect on init (with token) and be ready to send/receive. Show message history (by pulling last X messages from REST API and then live updating with websockets).
 - In doctor’s UI, similarly a chat view for each patient or an integrated messenger that lists all recent conversations. Perhaps a simple approach: doctor goes to a patient profile and there’s a chat tab to talk with that patient specifically (fits one-to-one model).
 - Also allow admin-to-doctor or admin broadcast? That’s less needed. Focus on patient-doctor messaging.

- Ensure the WebSocket auth (maybe pass JWT in query param or do a token check after upgrade) so only authorized users connect.
- If time doesn't permit full WS, we could do periodic polling for new messages, but WS is more real-time and efficient. We try for WS for learning purpose.
- **Email/SMS Integration for Reminders:** Now implement the actual sending of notifications that we scheduled in Sprint 3.
 - Set up a Twilio trial account, get API creds, verify a test phone number (likely our own). In Go, use Twilio's REST API to send SMS (use their Go client or just HTTP POST to Twilio's endpoint). Similarly, set up SendGrid (or we can use a simple SMTP to Gmail if that's easier with a throwaway account) for emails. But SendGrid free allows 100/day without credit card, which is enough. Use SendGrid's Go library to send templated emails (like "Hello {{name}}, this is a reminder to take ...").
 - Connect this in the reminder scheduler: when a reminder is due, call the appropriate send function. For safety, maybe have a config flag to disable actual sending in dev. In production (even on free tier), it should actually send. Because we might be limited in Twilio trial to certain numbers, we can initially put our number as the patient's number for testing. Document that for actual use, the project owner should upgrade Twilio or switch to email-only for more coverage.
 - Also implement appointment reminders similarly: if we have an Appointment table (we might create one now if not earlier). Appointment scheduling feature: we might allow doctors to record appointments and link to patients (with date/time). If we hadn't, we could do a simplified assumption that every record is an appointment that already happened. But ideally, an Appointment entity with date, patient, doctor, and status (scheduled/completed). If implemented, then doctors can add appointments and patients can request (depending on time). This sprint we can at least allow doctors to schedule and see them on a calendar.
 - Appointment Calendar (Doctor side): Possibly integrate a library or use fullcalendar.io (which has an Angular wrapper but might not be free? The core is free iirc). Or simply display a list grouped by date. Given time constraints, a fancy calendar UI is optional, but a basic one is beneficial. We plan minimal: a table of upcoming appointments and a form to add new one (select date/time and patient).
 - If time allows, a patient could see a list of available slots (from a preset schedule) and request one – but that might be too much to implement now. Instead, patient can just message doctor to request appointment as a simpler alternative.
- **Knowledge Base AI Integration (Phase 2):** If not done in Sprint 2 fully, here we integrate the vector search properly.
 - Deploy or enable Qdrant/pgvector. If Qdrant, perhaps set it up in Docker locally and connect. If using an external, integrate API. Ensure all existing docs are indexed: write a one-time script to embed all docs and upsert to vector DB. On each new or updated doc, also update the vector index.
 - Implement the "Similar Documents" feature: an API endpoint that given a doc returns top N similar docs (via vector search). Use cosine similarity. If using Qdrant, a simple query with dot or cosine metric is done. If using Postgres pgvector, use an ORDER BY vector <-> target limit 5.
 - Use that in the UI: On a knowledge doc page, show "Related documents:" listing those similar (title + maybe similarity score). This helps editors notice duplicates.
 - For conflict detection: we might not fully implement automated contradiction analysis (that might be researchy). But we can simulate a bit: if two docs are very similar but not the same, highlight that as conflict potential. Possibly use a simple text diff to see differences. If differences are major (like one says "Dose is 5mg" other says "10mg"), that's what we want to catch. Could write a regex to extract numbers/units and see if

mismatched – but that’s hacky. As a proof of concept, perhaps integrate a call to OpenAI (if allowed) to analyze two docs and return a verdict. But since we want open-source only ideally, we skip direct OpenAI calls (unless the user explicitly okays Azure which implies OpenAI).

- Alternatively, just list similar docs side by side for admin to manually inspect and decide conflict. That fulfills the spirit. Perhaps an admin page “Potential Conflicts” that simply shows pairs of doc titles flagged. The admin then edits as needed.
- **General Refinements:** Fix any known issues from earlier sprints. Improve UI/UX where possible (e.g. add loading spinners, form validations with clear error messages, etc.). Ensure the encryption and security measures (like if any missing auth checks) are tightened. Also verify the audit logging – we might add some logs in DB or at least server console for critical events.
- By end of Sprint 4, the platform should be nearly feature-complete: communications working, reminders actually sending (tested perhaps with a few examples), and knowledge base intelligence integrated. We’ll likely deploy updates to our staging environment (the free-tier hosts) regularly and test live as well.
- **Deliverables:** Chat feature working between a doctor and patient, automated reminders demonstrably working (e.g. an email received for a test patient), and overall system refined.

- **Sprint 5: Testing, Optimization, and Final Touches (Weeks 9-10)**

Goals: Finalize the project, do thorough testing (unit, integration, user acceptance), optimize performance, and add any extra usability features (calendars, print options, etc.) that were left. Aim to deploy the final version by April 22.

- **Testing & Bug Fixing:** Conduct end-to-end testing of all user flows: a patient signup -> login -> upload file -> get reminder; a doctor login -> create record -> prescription -> patient gets it; knowledge admin update doc -> see flags, etc. Use a small group of test users if available to try and provide feedback. Fix any bugs uncovered (security holes, crashes, logic errors). Write additional automated tests for crucial paths.
- **Performance Tuning:** On free tiers we have limited resources, so check memory usage of Go app (maybe tune GORM queries, use pagination on lists). Add indexes if queries are slow. For vector search, ensure that index is used or number of dimensions is not huge to slow down search. Possibly pre-compute some dashboard stats if they were heavy to compute on the fly (cache them for a session or so). But given small data initially, likely fine.
- **UI/UX Enhancements:** Add **doctor calendar view** if not yet: e.g. integrate a simple calendar widget to visually show appointment slots. Ensure all pages are responsive (test on mobile sizes via Chrome dev tools). Add the ability for doctors to **print a patient’s record** or download a PDF summary – we can use a library or simply rely on browser print of the page. For example, a “Print Record” button that opens a printable view (could just trigger `window.print()` for simplicity). Similarly, patients might want to print lab results or save them – since they can download their uploaded files, that’s covered; printing the prescription might be needed (we can generate a PDF of the prescription details with clinic info – maybe out of scope, but at least provide a nicely formatted HTML that can be printed).
- **Polish Notifications:** If any notifications in-app are needed (like a toast “Record saved successfully”), add those via an Angular library or simple service.
- **Documentation:** Prepare **clear diagrams** and documentation for the project (if not already in this guide). Perhaps create an architecture diagram image and an ERD diagram to include in the project docs (not necessarily in-app, but for the repo or report). Make sure code is well-commented, and provide a README with setup instructions (how to run locally, how to deploy to chosen free services).

- **Real-world usability improvements:** Possibly implement **export features:** e.g. allow admin to export the entire knowledge base to a CSV or patient list to Excel. Or allow patient to download all their data (for compliance). These are bonus if time allows.
- With everything tested and finalized, we'll deploy the final build to the production environment (still free-tier, but treat it as production). Monitor that deployment for any issues (like memory leaks or crashes – since free-tier often logs can be seen, we'll watch those).
- Finally, the project is delivered on (or before) April 22 with all requested features and additional improvements in place.

Throughout all sprints, we'll have **regular check-ins** to show progress to stakeholders, adjust scope if needed, and incorporate feedback early (agile approach). By breaking it down, we mitigate risk and ensure by Sprint 5 we're mostly polishing rather than doing big features.

Conclusion & Future Enhancements

By following this guide, we will build a **comprehensive medical monitoring platform** that blends knowledge management with patient health records, using a modern tech stack and strictly free or open-source resources. The design emphasizes maintainability (Angular for rich front-end, Go for high-performance back-end) and data integrity/security (SQL DB with structured schema, RBAC, encryption).

We have incorporated all requested features – from document health checks (with refresh timers for stale docs ² and duplicate detection) to doctor/patient workflows and admin oversight – and even added usability enhancements like scheduling and messaging to simulate a real EMR system. The use of open-source libraries and models ensures that the solution can be run without licensing costs, and hosting on free tiers makes it practically zero-cost to deploy initially.

Diagrams & Flows: (In the project documentation, we will include diagrams such as an architecture diagram showing Angular client, Go API, DB, etc., and an ERD illustrating tables like Patient, Doctor, Record, Prescription, KnowledgeDoc, etc., with relationships as described. These aid in understanding and onboarding developers.) The modular breakdown of components and clear interface definitions (API endpoints, data contracts in JSON) will make implementation smoother and allow parallel development (e.g. one developer can work on knowledge module while another on patient module). Reusable modules (like a generic List component, or a confirmation dialog service) will be utilized in the front-end to speed up development.

Real-world Usability: Features like calendar views, upload history logs, and print-friendly pages will improve day-to-day usage for doctors and patients. For example, a doctor can quickly glance at their schedule on a calendar and click an appointment to view that patient's record – streamlining workflow. Patients can maintain a personal health file with all their uploads and easily share or print them if visiting an external specialist. These additional touches move the project beyond a basic demo to a more **production-ready system**.

Finally, while our platform is scoped for a single clinic or small environment (due to free-tier limits), the architecture can evolve: it could be containerized and scaled to a cloud cluster, and more features (like telehealth video calls or advanced AI diagnostics) could be integrated in the future. But even in its initial form, it provides a solid, secure, and feature-rich solution for medical and document health monitoring using entirely free and open-source technology.

References: (The implementation plan and feature design was informed by best practices and examples from open-source EMR systems and knowledge management resources: we considered features from OpenEMR ⁸ ⁹ ¹⁵ , and applied knowledge base upkeep tips such as regular reviews and flagging outdated info ² ⁴ . The choice of tools and services was guided by documentation and community info about free tiers ³¹ ³⁰ ²² and the capabilities of open libraries for search and embeddings ¹⁹ ²⁴ .)

¹ ³ ⁴ How to Document Tips for IT Knowledge Bases | NinjaOne

<https://www.ninjaone.com/blog/how-to-document-tips-for-it-knowledge-bases/>

² Keeping a Knowledge Base Up to Date is a Nightmare : r/sysadmin

https://www.reddit.com/r/sysadmin/comments/1ikvmm9/keeping_a_knowledge_base_up_to_date_is_a_nightmare/

⁵ ⁶ ¹¹ ¹² KB STATISTICS - General Bots Documentation

<https://docs.pragmatismo.com.br/06-gbdialog/keyword-kb-statistics.html>

⁷ ⁸ ⁹ ¹⁵ OpenEMR Features - OpenEMR Project Wiki

https://www.open-emr.org/wiki/index.php/OpenEMR_Features

¹⁰ Twilio Free Trial Limitations - Twilio Help Center

<https://help.twilio.com/articles/360036052753-Twilio-Free-Trial-Limitations>

¹³ Delay in Treatment: Failure to Contact Patient Leads to Significant Complications | PSNet

<https://psnet.ahrq.gov/web-mm/delay-treatment-failure-contact-patient-leads-significant-complications>

¹⁴ RBAC

<https://www.casbin.org/docs/rbac>

¹⁶ HIPAA Encryption Requirements... Safeguard Data in Transit!

<https://www.enguard.com/hipaa-encryption-requirements/>

¹⁷ HIPAA Encryption Requirements: An Updated Guide - Drata

<https://drata.com/blog/hipaa-encryption-requirements>

¹⁸ Fortify Your Data: A Practical Guide to Encryption with Go | by Codeclowns | . | Medium

<https://medium.com/@codeclowns01/fortify-your-data-a-practical-guide-to-encryption-with-go-707cf7700581>

¹⁹ ²⁰ ²¹ ³⁴ GitHub - kelindar/search: Go library for embedded vector search and semantic embeddings using llama.cpp

<https://github.com/kelindar/search>

²² ²³ ²⁸ ²⁹ Qdrant vs Pinecone: Vector Databases for AI Apps - Qdrant

<https://qdrant.tech/blog/comparing-qdrant-vs-pinecone-vector-databases/>

²⁴ Utilizing ChatGPT as a scientific reasoning engine to differentiate conflicting evidence and summarize challenges in controversial clinical questions - PMC

<https://pmc.ncbi.nlm.nih.gov/articles/PMC11187493/>

²⁵ ²⁶ Pricing - Azure AI Search | Microsoft Azure

<https://azure.microsoft.com/en-us/pricing/details/search/>

²⁷ Holori - The Ultimate Guide to OpenAI Pricing: Maximize Your AI investment

<https://holori.com/openai-pricing-guide/>

³⁰ ³¹ GitHub - ripienaar/free-for-dev: A list of SaaS, PaaS and IaaS offerings that have free tiers of interest to devops and infradev

<https://github.com/ripienaar/free-for-dev>

32 33 **Generating Document Embeddings in Go | CodeSignal Learn**

<https://codesignal.com/learn/courses/document-processing-and-retrieval-with-langchaingo/lessons/generating-document-embeddings-in-go>